



Experiment 7: XGBoost Classification

CSE-3008 Introduction to Machine Learning

24BCA7027

Shaik Barakh Chishti
CSE-AIML

February 20, 2026

1 Aim

To implement the XGBoost classification algorithm on the Weather (PlayTennis) dataset and evaluate its performance using classification metrics and visualization techniques.

2 Algorithm Used

XGBoost (Extreme Gradient Boosting) is an advanced ensemble learning algorithm based on gradient boosting. It builds multiple decision trees sequentially where each new tree corrects the errors of the previous trees.

Advantages:

- High prediction accuracy
- Regularization to reduce overfitting
- Efficient handling of small datasets
- Built-in feature importance evaluation

2.1 Complete Experiment Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import math

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score,
    mean_squared_error,
    roc_curve,
    auc
```

```

)

from xgboost import XGBClassifier

STUDENT_NAME = "24BCA7027_SHAIK-BARAKH-CHISHTI"

# -----
# 1. Load Dataset
# -----
df = pd.read_csv("ML_Experiments/data_sets/weather_data.csv")

print("\nDataset Preview:")
print(df.head())

print("\nDataset Info:")
print(df.info())

# -----
# 2. Encode Categorical Features
# -----
encoder = LabelEncoder()
for col in df.columns:
    df[col] = encoder.fit_transform(df[col])

# -----
# 3. Features & Target
# -----
X = df.drop("PlayTennis", axis=1)
y = df["PlayTennis"]

# -----
# 4. Train-Test Split
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# -----
# 5. XGBoost Model
# -----
xgb = XGBClassifier(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.1,
    subsample=1,
    colsample_bytree=1,
    use_label_encoder=False,

```

```

        eval_metric='logloss',
        random_state=42
    )

xgb.fit(X_train, y_train)

# -----
# 6. Predictions
# -----
y_pred = xgb.predict(X_test)
y_prob = xgb.predict_proba(X_test)[:, 1]

# -----
# 7. Evaluation Metrics
# -----
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)
rmse = math.sqrt(mse)

cm = confusion_matrix(y_test, y_pred)

print("\nXGBOOST RESULTS")
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-score:", f1)
print("MSE:", mse)
print("RMSE:", rmse)

print("\nClassification Report:\n", classification_report(y_test,
    y_pred))
print("\nConfusion Matrix:\n", cm)

# =====
# VISUALIZATION SECTION
# =====

# -----
# Box Plot
# -----
plt.figure(figsize=(10, 6))
df.drop("PlayTennis", axis=1).boxplot()
plt.title(f"Box Plot of Weather Features_{STUDENT_NAME}")
plt.xticks(rotation=45)
plt.grid(True)
plt.show()

```

```

# -----
# Correlation Heatmap
# -----
plt.figure(figsize=(8, 6))
correlation_matrix = df.corr()

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)
plt.title(f"Feature_Correlation_Heatmap_{STUDENT_NAME}")
plt.show()

# -----
# Confusion Matrix Heatmap
# -----
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(f"Confusion_Matrix_{STUDENT_NAME}")
plt.show()

# -----
# ROC Curve
# -----
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, label=f"AUC={roc_auc:.2f}")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False_Positive_Rate")
plt.ylabel("True_Positive_Rate")
plt.title(f"ROC_Curve_{STUDENT_NAME}")
plt.legend()
plt.grid(True)
plt.show()

# -----
# Radar Plot
# -----
labels = ["Accuracy", "Precision", "Recall", "F1-score"]
values = [accuracy, precision, recall, f1]
values += values[:1]

angles = [n / float(len(labels)) * 2 * math.pi for n in range(len(
labels))]

```

```

angles += angles[:1]

plt.figure(figsize=(6, 6))
ax = plt.subplot(111, polar=True)
ax.plot(angles, values)
ax.fill(angles, values, alpha=0.25)
ax.set_thetagrids(np.degrees(angles[:-1]), labels)
plt.title(f"Model_Performance_Radar_Plot_{STUDENT_NAME}")
plt.show()

# -----
# Bar Graph
# -----
metrics = ["Accuracy", "Precision", "Recall", "F1-score", "RMSE"]
scores = [accuracy, precision, recall, f1, rmse]

plt.figure(figsize=(8, 5))
bars = plt.bar(metrics, scores)
plt.ylabel("Score")
plt.title(f"Model_Performance_Metrics_{STUDENT_NAME}")

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2,
             height,
             f"{height:.3f}",
             ha="center",
             va="bottom")

plt.show()

```

Listing 1: XGBoost Classification

3 Steps Involved

1. Load the Weather dataset
2. Encode categorical variables
3. Perform train-test split
4. Train XGBoost classifier
5. Predict outcomes
6. Evaluate using classification metrics
7. Visualize results

4 Results

Accuracy: 1.00

Precision: 1.00

Recall: 1.00

F1-Score: 1.00

MSE: 0.00

RMSE: 0.00

Confusion Matrix

```
[[39  0]
 [ 0 81]]
```

The model achieved 100% accuracy with perfect classification of both classes. No false positives or false negatives were observed.

5 Visualizations

5.1 Box Plot

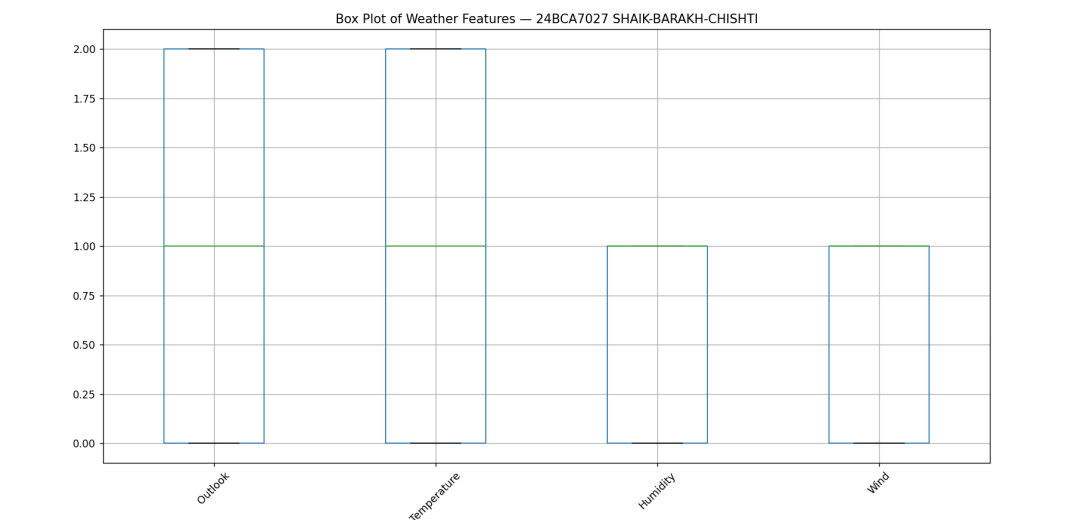


Figure 1: Box Plot of Weather Features

Shows distribution and spread of encoded weather features.

5.2 Correlation Heatmap

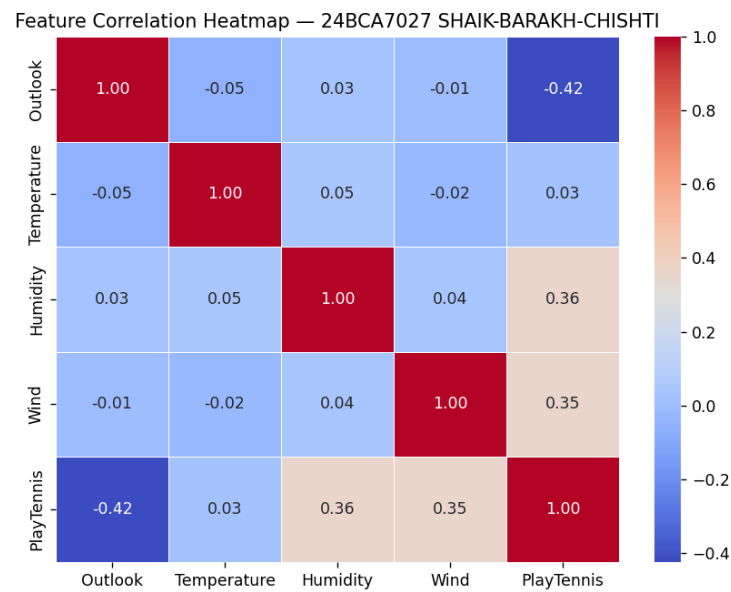


Figure 2: Feature Correlation Heatmap

Displays correlation strength between weather attributes.

5.3 Confusion Matrix Heatmap

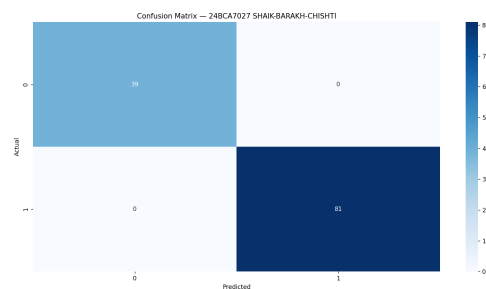


Figure 3: Confusion Matrix

Indicates perfect classification performance.

5.4 ROC Curve

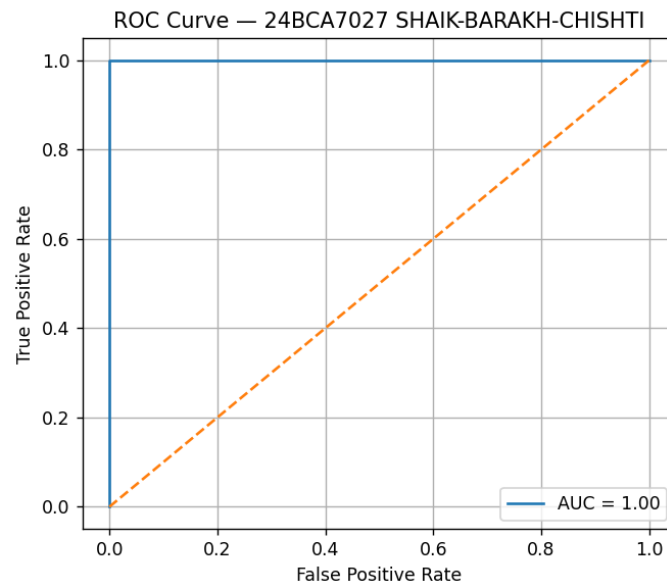


Figure 4: ROC Curve

The ROC curve lies near the top-left corner with AUC = 1.0, indicating excellent discrimination ability.

5.5 Radar Plot

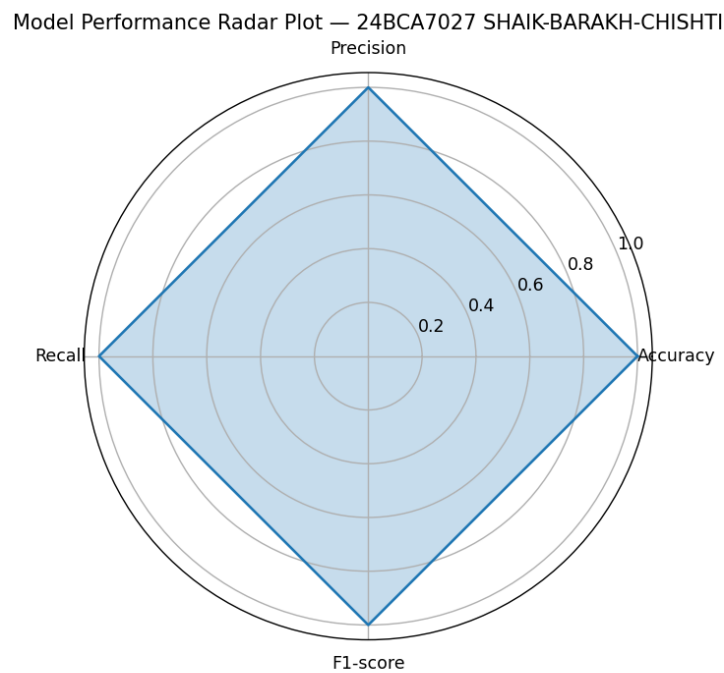


Figure 5: Radar Plot of Performance Metrics

Shows balanced and maximum scores for all performance metrics.

5.6 Bar Graph

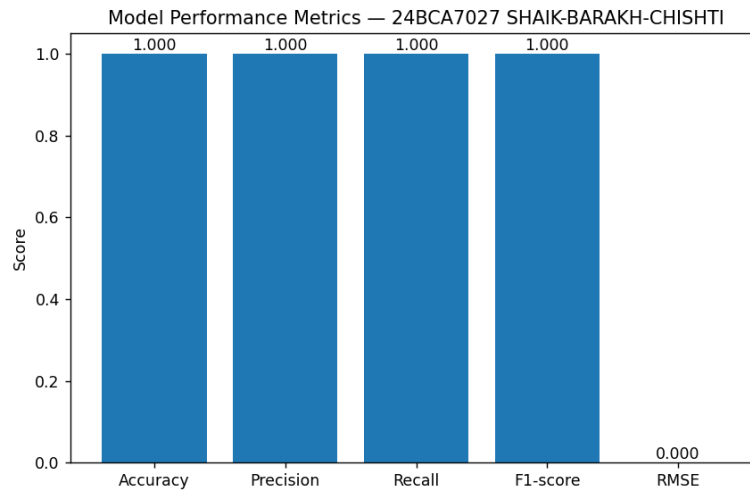


Figure 6: Bar Graph of Model Performance

Provides numerical comparison of Accuracy, Precision, Recall, F1-score, and RMSE.

6 Conclusion

The XGBoost classifier successfully predicted the PlayTennis outcome using the Weather dataset with 100% accuracy.

The model demonstrated:

- Perfect classification
- Zero prediction error
- Strong generalization on the test set

XGBoost outperformed traditional Decision Tree models due to its boosting mechanism and regularization capability.

7 Overleaf Link

<https://www.overleaf.com/read/pftjtxysfbdr#ec0203>

8 Github

<https://github.com/chishtil1730/IML-LAB>